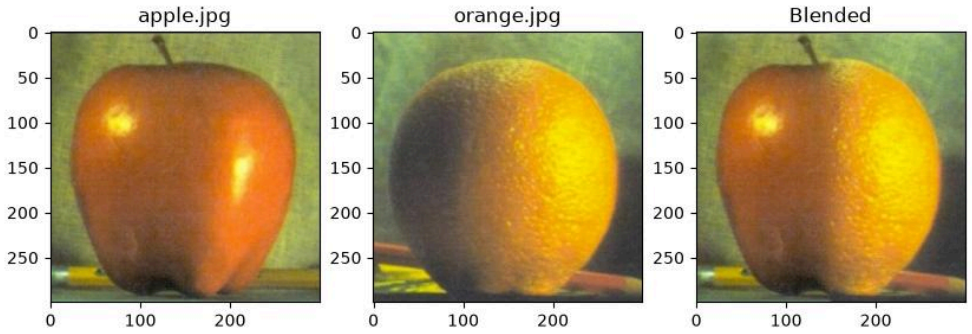
Project 2: Fun with Filters and Frequencies!



Multi-resolution blending with Laplacian and Gaussian stacks: Oraple

Part 1.1: Convolutions from Scratch

A convolution takes a filter (AKA kernel) and slides it across an image. At every position, the filter is multiplied element-wise with the pixels values of the image that it currently overlaps with.

I implemented a convolution from scratch using NumPy! My homemade convolution:

- Calculates how many zero's to pad on the top, bottom, left, and right of the input image such that the output of the convolution has the same dimensions as the input image (this is known as "same" padding!)
- Flips the filter
- Implements the convolution as a quadruple for-loop
- And implements the convolution as a double for-loop by taking advantage of NumPy's vectorized operations!

Contents

Part 1.1: Convs from Scratch
Part 1.2: Finite Difference Operators
Part 1.3: Derivative of Gaussians Part
2.1: Image Sharpening
Part 2.2: Hybrid Images
Part 2.3: Gaussian and Laplacian Stacks
Part 2.4: Multi-resolution Blending

As a very satisfying sanity check, I compared my hand-crafted convolution with `scipy.signal.convolve2d` to verify that my implementation produced the exact same convolution outputs :)

I handled the boundaries of my input images by same-padding with zeros so that the output would have the same dimensions as the input. I calculated the right amount of padding to add on each side using the formula to calculate the dimensions of a convolution, assuming stride (S) = 1, and finding the value of P that set D_out = D_in for both the width and height.

$$D_{out} = \lfloor \frac{D_{in} - K + 2P}{S} \rfloor + 1$$

The runtime of my double for-loop is much faster than my quadruple for-loop, but both are outpaced by SciPy's convolution implementation! SciPy's implementation includes optimizations like automatically switching between direct computations or a Fast Fourier Transform method and using a compiled C/Fortran backend instead of Python's interpreter for small kernels. Clearly there's a lot of engineering rabbit-holes to dive down to learn how to further optimize my handmade convolution!

```
def convolve_2d(matrix, filter, quad_for_loop=False):
    """Part 1.1: Convolutions from Scratch!"""
    h, w = matrix.shape

    if len(filter.shape) < 2:
        # Reshape row vector into (d, 1) matrix
        filter = np.expand_dims(filter, axis=0)
    fh, fw = filter.shape

    # D_out = np.floor((D_in - K + 2P) / S) + 1
    # P = ((D_out - 1) * S - D_in + K) / 2
    # Pad input image using same-padding s.t. output shape = input shape
    # Fill with constant value = 0
```

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending

```
    y_pad = (fh - 1) // 2
    x_pad = (fw - 1) // 2
    padded = np.pad(
        matrix, ((y_pad, y_pad), (x_pad, x_pad)), mode="constant", constant_values=0
    )

    flip_filter = np.flip(filter)

    out = np.zeros((h, w))

    if quad_for_loop:
        # Quadruple for-loop implementation
        for oy in range(h):
            for ox in range(w):
                for fy in range(fh):
                    for fx in range(fw):
                        out[oy, ox] += flip_filter[fy, fx] * padded[oy + fy, ox + fx]
    else:
        # Double for-loop implementation
        for oy in range(h):
            for ox in range(w):
                out[oy, ox] = np.sum(
                    flip_filter * padded[oy : oy + fh, ox : ox + fw], axis=None
                )

    # Compare with scipy.signal.convolve2d
    scipy_out = scipy.signal.convolve2d(matrix, filter, mode="same", fillvalue=0)
    assert np.allclose(out, scipy_out, atol=1e-8), (
        "Convolution does not match scipy.signal.convolve2d"
    )

    return out
```

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending

I used my homemade convolution to convolve a grayscale image of myself with 3 different kernels/filters:

- **Dx:** a finite difference filter that detects changes in the x-direction (vertical edges)

$$D_x = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

- **Dx:** a finite difference filter that detects changes in the y-direction (horizontal edges)

$$D_y = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

- **Box filter:** an averaging filter that computes the average of all the pixels in a 9x9 neighrborhood.

$$B = \frac{1}{81} \begin{bmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{bmatrix}$$

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

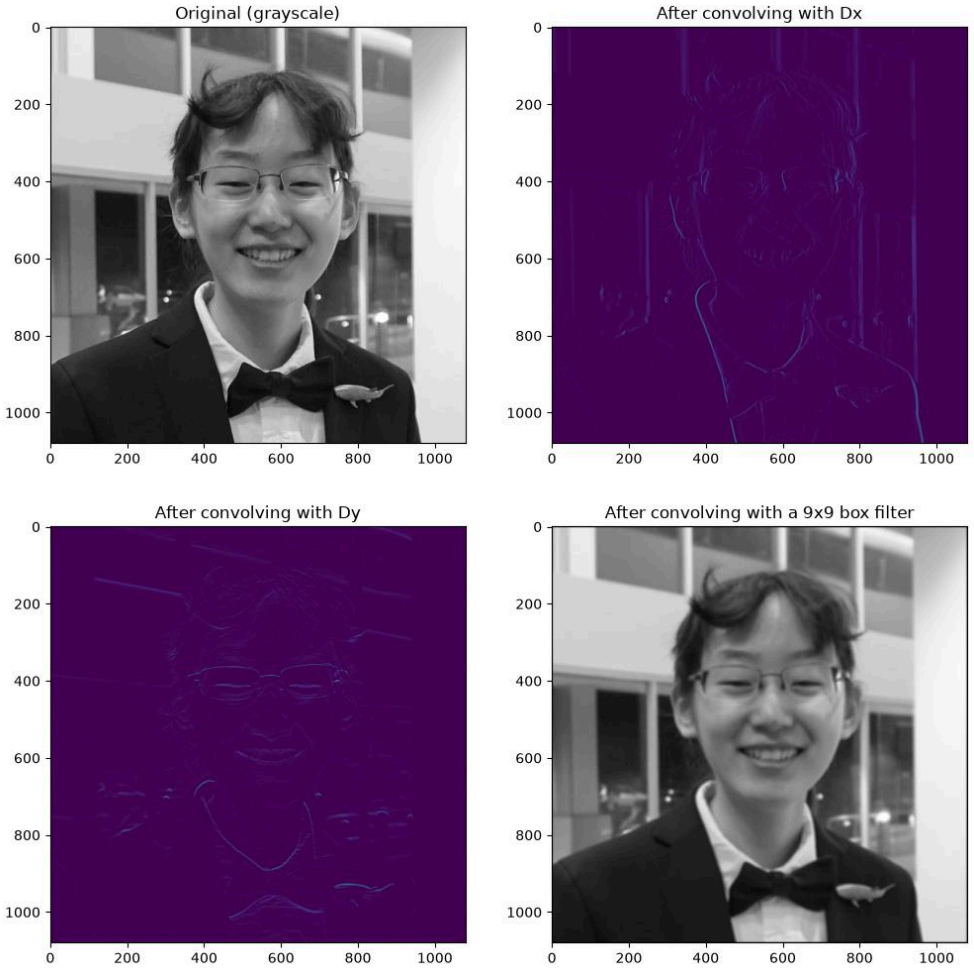
Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending



Convolving a picture of myself with Dx, Dy, and a 9x9 box filter

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

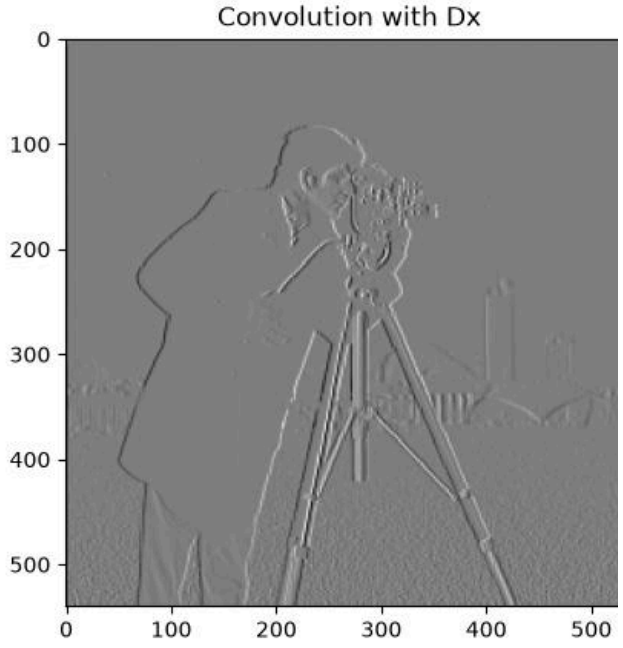
Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

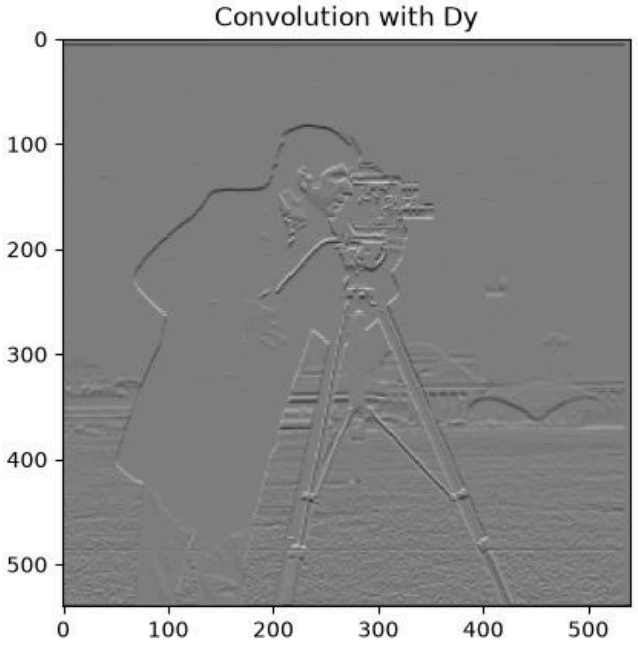
Part 2.4: Multi-resolution Blending

Part 1.2: Finite Difference Operators

By convolving an image with Dx or Dy, we can detect different kinds of edges! Convolving an image with Dx detects vertical edges (i.e. changes in intensity between horizontally-adjacent pixels). Likewise, convolving an image with Dy detects horizontal edges (i.e.changes in intensity between vertically-adjacent pixels).



Convolving camerman.jpg with Dx



Convolving camerman.jpg with Dy

The gradient is a vector of partial derivatives. Therefore, the gradient of an image at each coordinate is a vector containing the corresponding pixels in the response after convolving the image with Dx and Dy. Taking the magnitude of the gradient measures edge strength for all edge orientations in the image. To better visualize the edges, I also binarized them: I identified all coordinates where the magnitude of the gradient at that position was above a certain threshold (found through visual trial and error), then mapped those pixels to 1 and masked all other pixels to 0.

Picking a lower threshold for binarizing edges meant that a greater proportion of edges had strengths above that threshold – so the result had more noise. Picking a higher threshold for binarizing edges meant only very strong edges showed. But this sometimes meant that background details or soft edges were omitted. For instance, in cameramn.jpg, choosing a high enough threshold such that the result didn't have noisy grass meant the edges of the background tower were also usually discarded.

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

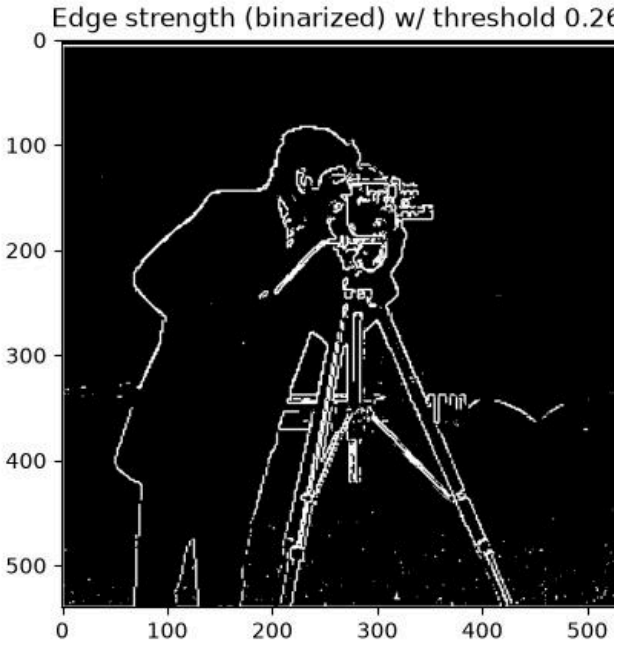
Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

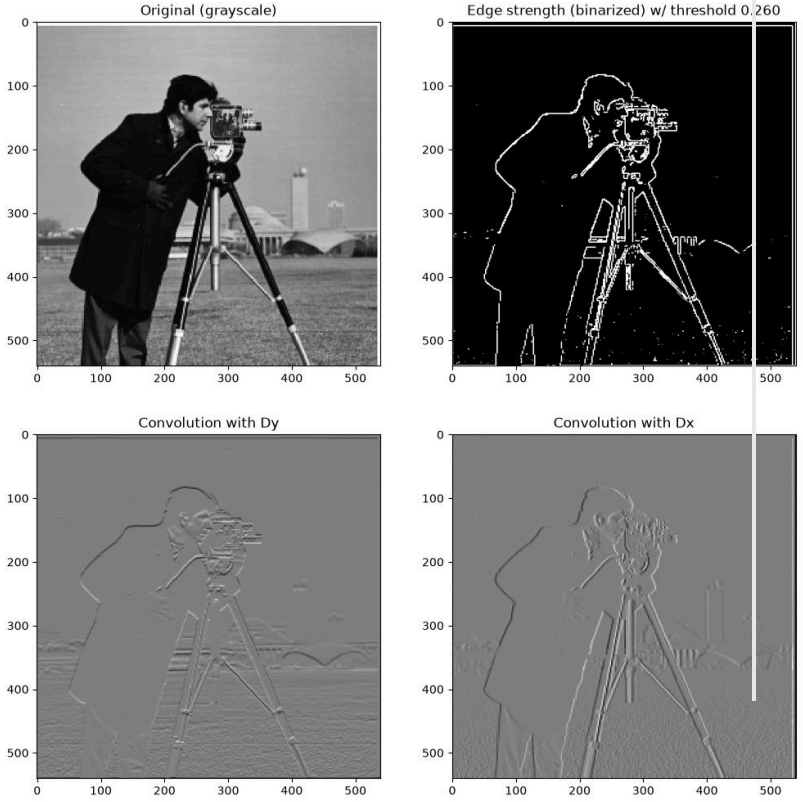
Part 2.4: Multi-resolution Blending

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$$

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$



Edge strengths (binarized) of cameraman.



Comparing the results of convolving cameraman.jpg with finite difference operators

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

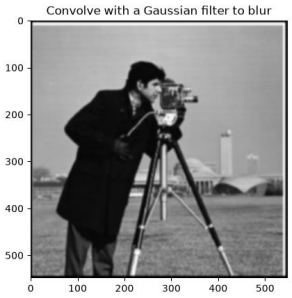
Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

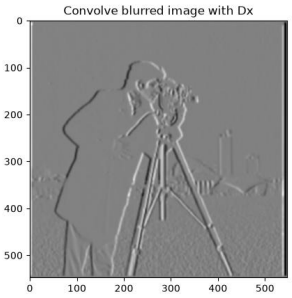
Part 2.4: Multi-resolution Blending

Part 1.3: Derivative of Gaussian Filter

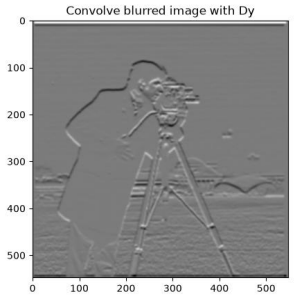
Directly convolving the image with finite difference operator filters can result in noisy edges. To reduce noisiness, I first blurred camerman.jpg by convolving it with a Gaussian filter before convolving it with Dx and Dy.



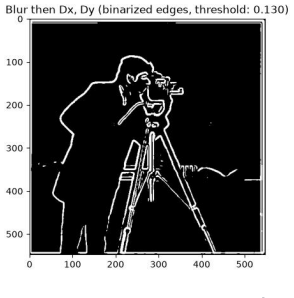
Step 1: blur the image by convolving with a Gaussian filter



Step 2A: convolve the blurred image with Dx



Step 2B: convolve the blurred image with Dy



Step 3: Compute the magnitude of the gradients and binarize edges

For each of Dx and Dy, this requires convolving the image with 2 separate filters (convolve with a Gaussian, then convolve again with either Dx or Dy). Based on the derivative theorem of convolution, the same process can be implemented by convolving the image with a single filter: a derivative of a Gaussian (DoG). To produce this filter, I first convolved a Gaussian filter with Dx and Dy respectively.

$$\frac{\partial}{\partial x}(h * f) = \left(\frac{\partial}{\partial x} h\right) * f$$

This is what taking the derivative of a 9x9 Gaussian looks like! AKA convolving the Gaussian kernel with the Dx kernel and the Dy kernel.

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

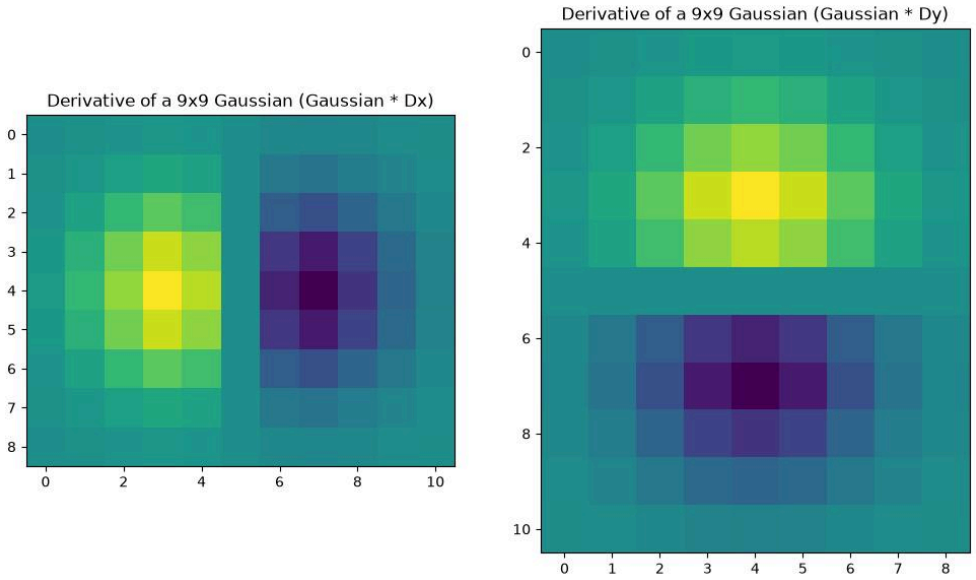
Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

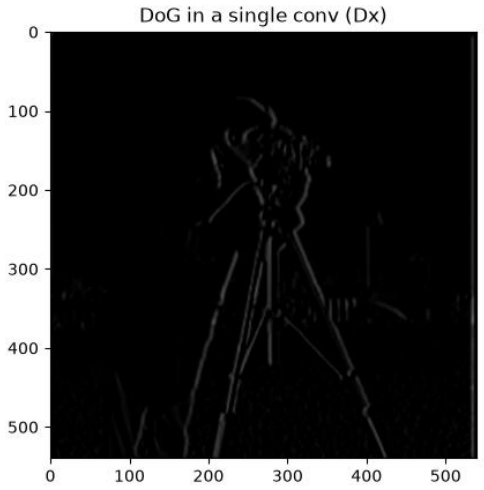
Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending

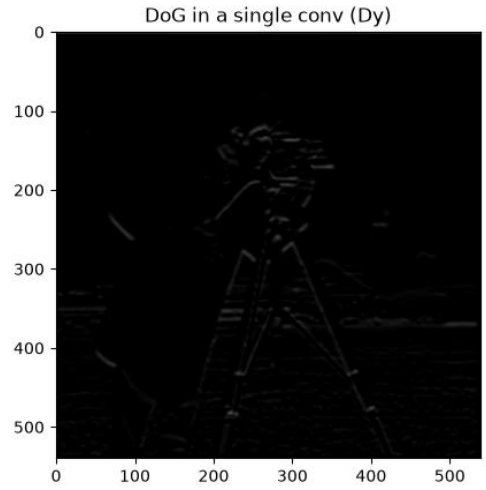


Taking the derivative of a Gaussian by convolving the Gaussian kernel with Dx and Dy

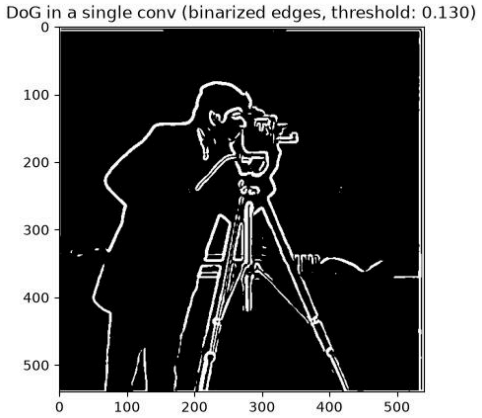
And these are the results of convolving the same cameraman.jpg image with the single-step DoG filters for detecting vertical and horizontal edges, then calculating its gradient magintidue and binarizing edges.



Single-conv DoG filter to detect vertical edges



Single-conv DoG filter to detect horizontal edges



Binarized edges from the single-conv DoG filters

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

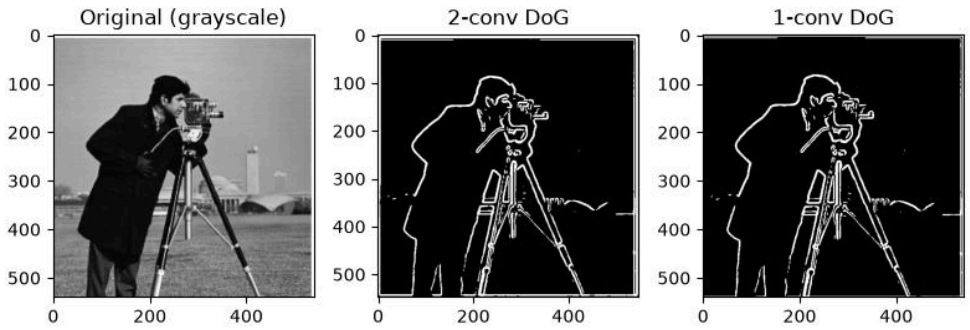
Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending



Comparing two-conv versus single-conv DoG filter results

Part 2.1: Image Sharpening

Applying (convolving) a Gaussian kernel to an image produces a blurrier version of that image – the Gaussian kernel acts as a low-pass filter such that the result only contains the low frequencies of the image. Subtracting the blurred version from the original image gives us the image's high frequencies. We can then add the high frequencies back to the original image (sometimes scaled by some α to control the "sharpness") to make it look sharper!

This sequence of operations – blur an image using a Gaussian kernel, subtract the blurred image from the original image to get the high frequencies, then adding the high frequencies back to the original image to create a sharpened version – can be combined into a single convolution called the unsharp mask filter (AKA the sharpening filter). In the following formula for the sharpening filter, f represents the original image, g represents the Gaussian kernel, and e is the unit impulse (essentially an identity kernel whose response is the same input it is convolved with). Alpha is a scalar value that determines how "sharp" the result looks.

$$f + \alpha(f - f * g) = f * ((1 + \alpha)e - \alpha g)$$

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

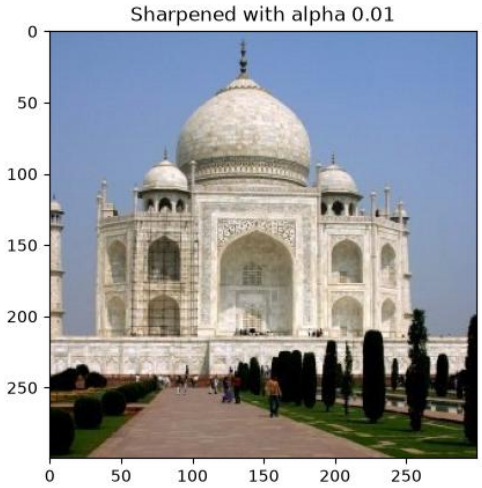
Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

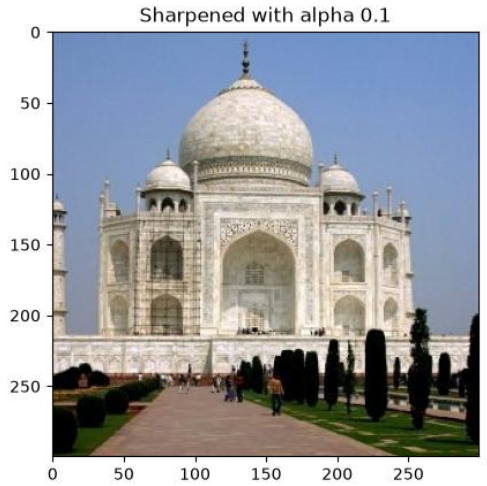
Part 2.4: Multi-resolution Blending



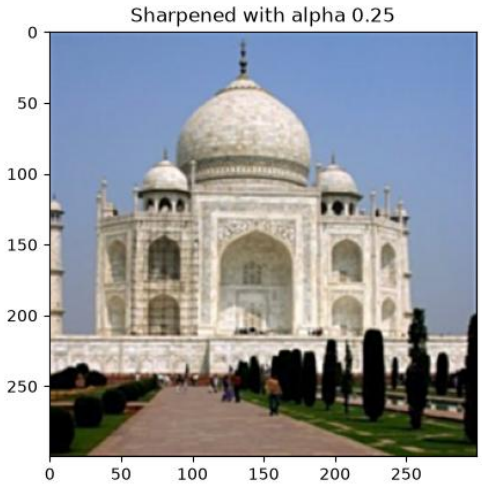
taj.jpg (original)



taj.jpg sharpened with a single-conv sharpening mask (α = 0.01)



taj.jpg sharpened with a single-conv sharpening mask (α = 0.1)



taj.jpg sharpened with a single-conv sharpening mask (α = 0.25)

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

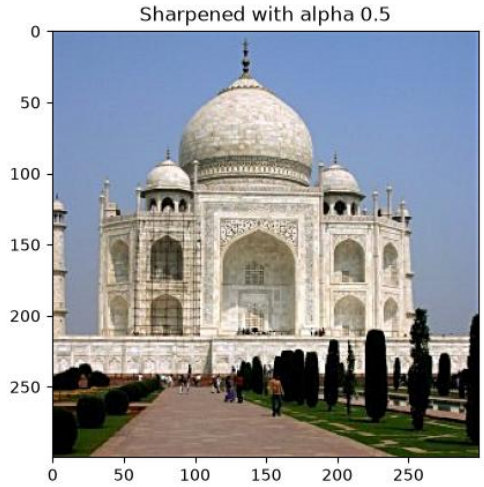
Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

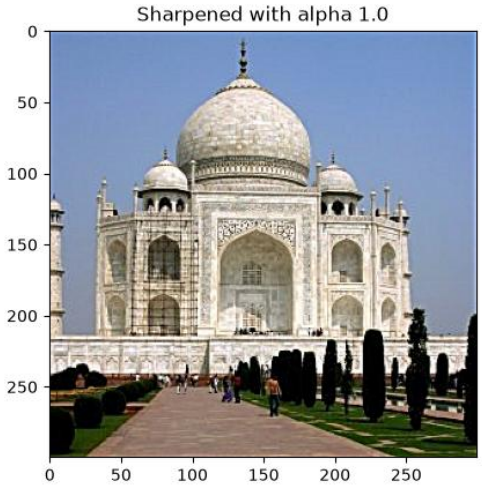
Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

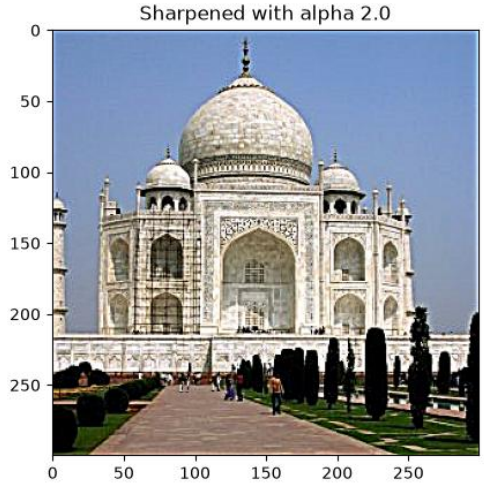
Part 2.4: Multi-resolution Blending



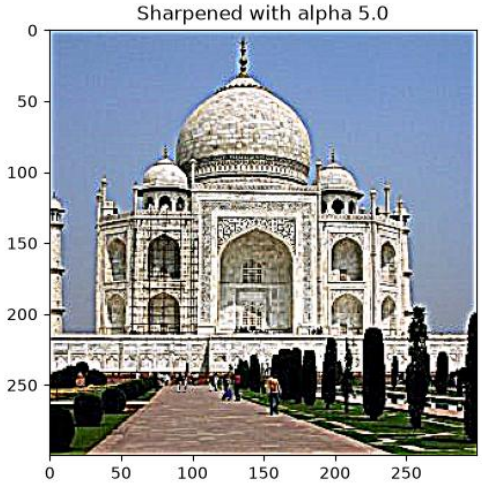
taj.jpg sharpened with a single-conv sharpening mask (α = 0.5)



taj.jpg sharpened with a single-conv sharpening mask (α = 1.0)



taj.jpg sharpened with a single-conv sharpening mask (α = 2.0)



taj.jpg sharpened with a single-conv sharpening mask (α = 5.0)

Here are some additional images of my own that I tried sharpening!

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending



clownfish.jpg (original)



clownfish.jpg sharpened with a single-conv sharpening mask (α = 1.0)



clownfish.jpg sharpened with a single-conv sharpening mask (α = 2.0)



chinatown.jpg (original)



chinatown.jpg sharpened with a single-conv sharpening mask (α = 1.0)



chinatown.jpg sharpened with a single-conv sharpening mask (α = 2.0)

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending



tetons.jpg (original)



tetons.jpg sharpened with a single-conv sharpening mask (α = 1.0)

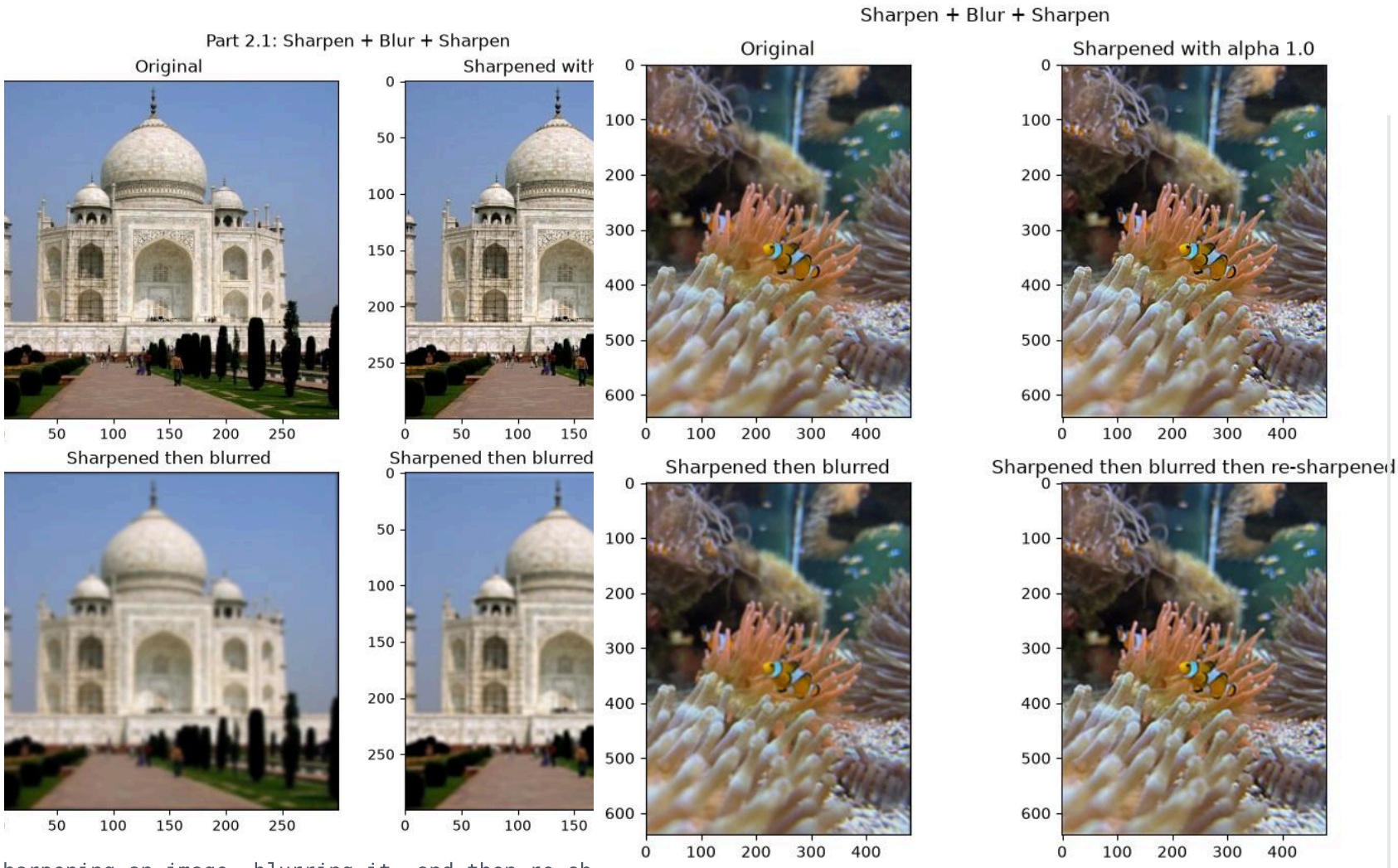


tetons.jpg sharpened with a single-conv sharpening mask (α = 2.0)

As an additional experiment, I also tried picking a sharp image, blurring it, then re-sharpening it. Blurring then re-sharpening didn't seem to recover the original image. Blurring discards some of the fine details in the original image. Applying the sharpening mask boosts the high-frequency details remaining in the blurred image, but it can't recover the information that was lost. Moreover, in the real world, rounding and numerical representation can make it impossible to reconstruct image data after it's been transformed.

Contents

- Part 1.1: Convs from Scratch
- Part 1.2: Finite Difference Operators
- Part 1.3: Derivative of Gaussians Part
- 2.1: Image Sharpening
- Part 2.2: Hybrid Images
- Part 2.3: Gaussian and Laplacian Stacks
- Part 2.4: Multi-resolution Blending



harpening an image, blurring it, and then re-sharpening it
.n does not recover the original image or the sharpened image.

Original, sharpened, blurred, and re-sharpened images of clownfish.jpg

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

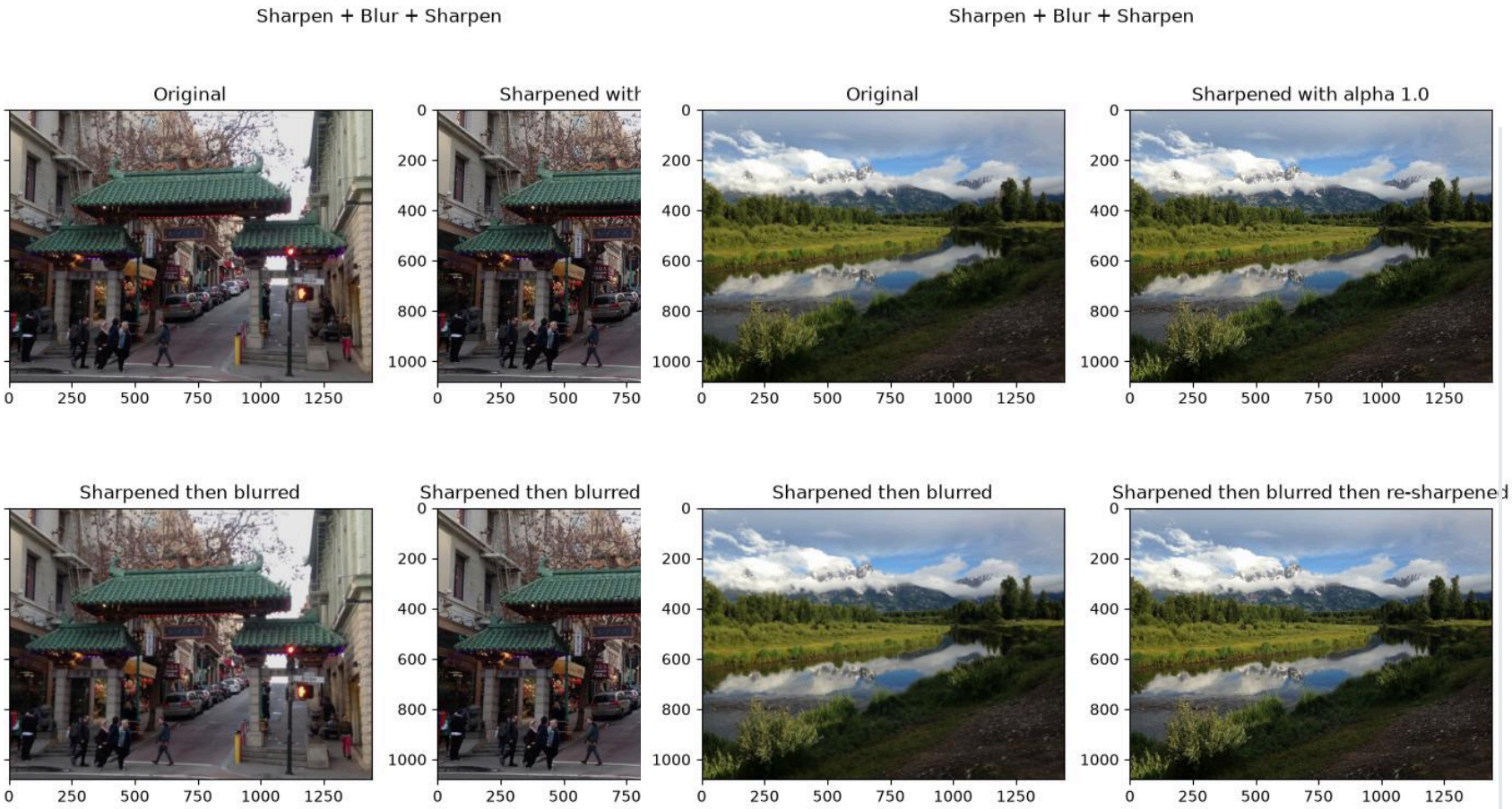
Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending



Original, sharpened, blurred, and re-sharpened images of chinatown.jpg

Original, sharpened, blurred, and re-sharpened images of tetons.jpg

Part 2.2: Hybrid Images

Our visions don't perceive the full range of frequencies equally at different distances. When we are closer to an image, we are able to pick up higher frequencies: precise details and fine textures. When we are further away from an image, only low frequencies in an image are perceivable: broad patterns and smoother textures. Hybrid images play on this phenomena!

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending

To create a hybrid image, I took two images but blended only specific frequencies from each one into a single image. I first selected 2 points in each image to use to align them together. Then I applied a high-pass filter to the first image, keeping only its high frequencies. For the high-pass filter, I subtracted a blurred version of the first image from its original image. And, I applied a low-pass filter to the second image, keeping only the low frequencies of that image. For the low-pass filter, I convolved the image with a 2D Gaussian kernel. I picked the cut-off frequency for each filter (i.e. the lower limit on which high frequencies to keep from the first image and a separate lower-limit on which low frequencies to keep from the second image) via visual experimentation. Lastly, I layered the two filtered images on top of each other based on the alignment. As a result, you may perceive the hybrid image as different things depending on your viewing distance.

Hybrid image 1: a human named Derek and a cat named Nutmeg

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

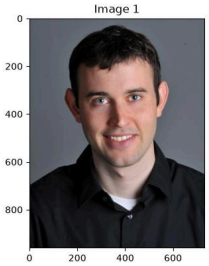
Part 2.4: Multi-resolution Blending

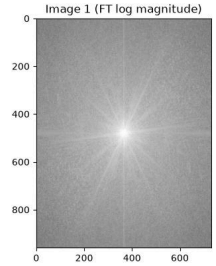


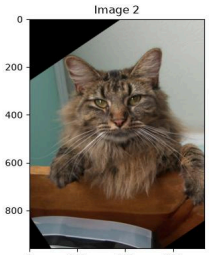
Derek (kept the high frequencies)

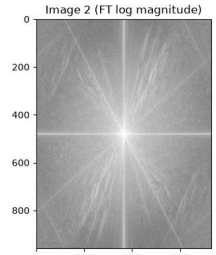


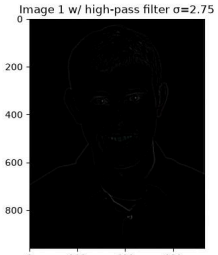
Derek's cat Nutmeg (kept the low frequencies)

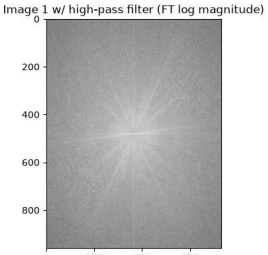








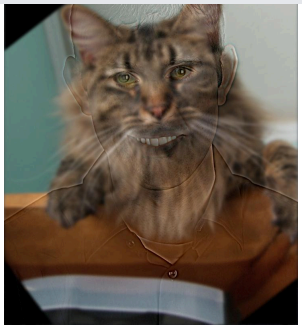




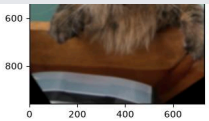


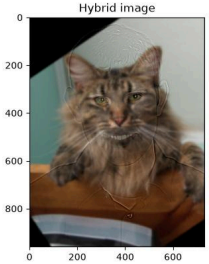


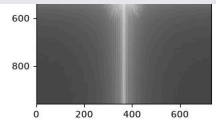
Deena's CS 180 Portfolio Project 0 Project 1 Project 2 Project 3 Project 4 Final Project

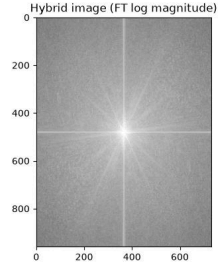


Hybrid image of Derek and Nutmeg. Up close, you can









Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3:

Part 2.4: Multi-resolution Blending

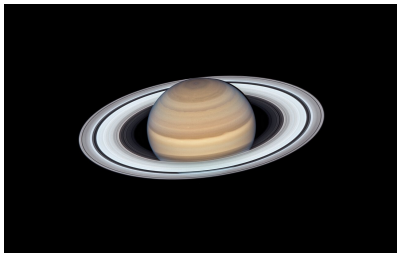
see Derek! From far away, it's almost all Nutmeg

Plots of the log magnitude of the Fourier transforms for each image

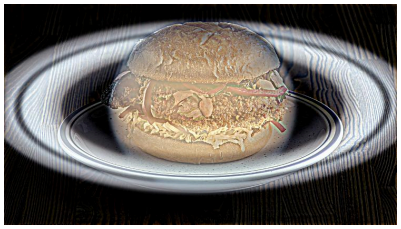
Hybrid image 2: a RT Rotisserie burger and Saturn



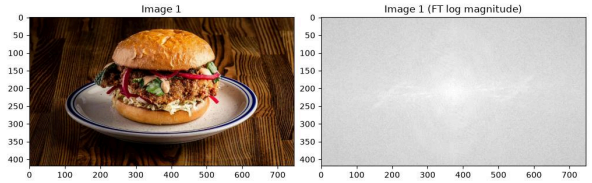
A RT Rotisserie burger (kept the high frequencies)

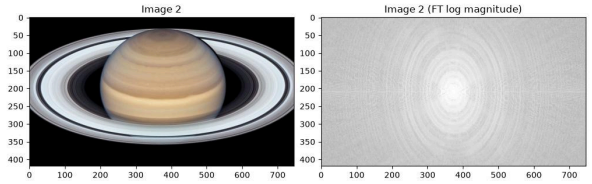


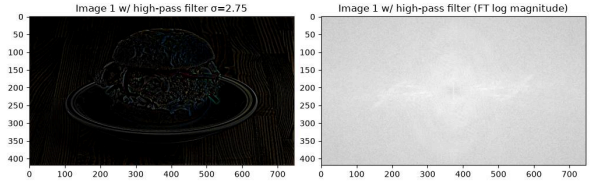
Saturn (kept the low frequencies)

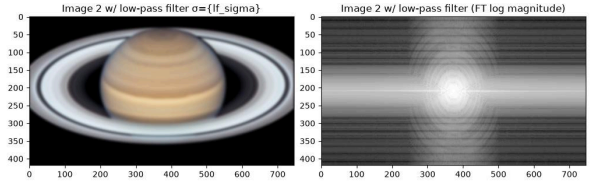


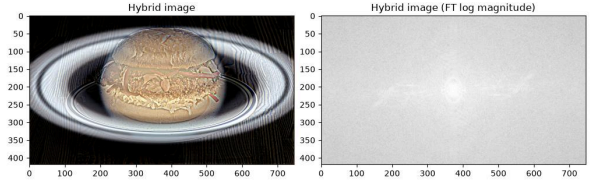
Hybrid image of a burger and Saturn. Up close, you can see the burger! From far away, it's mostly Saturn











Plots of the log magnitude of the Fourier transforms for each image

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending

Part 2.3: Gaussian and Laplacian Stacks

Gaussian and Laplacian stacks are ways to iteratively apply a Gaussian or Laplacian transformation to the same image without actually changing the image's dimensions. An image pyramid downsamples between each level, so the images at each level get smaller in smaller. In a stack, the image dimensions don't change from level to level. One advantage of this is that all the levels of a stack can then be saved into one big array of matrices! To create successive levels of a Gaussian stack without downsampling, we can use a larger Gaussian kernel ("larger" in the sense that it has a bigger sigma value and therefore also a bigger kernel size) from one level to the next. I.e. by doubling the value of sigma (and Gaussian kernel's size) at each level, the Gaussian stack behaves similarly to a Gaussian pyramid that downsamples to 1/2 the size at each level.

The Laplacian stack can be created by subtracting adjacent levels in the Gaussian stack. In this way, we can think of L_i as representing the information lost when going from the ith level in the Gaussian stack to the (i + 1)-th level. Since convolving Gaussian kernels with an image is like applying a low-pass filter, and applying Gaussian kernels of different sigmas/sizes means different cut-offs for what the range of frequenices can pass through are, L_i captures a band-pass of frequencies in between the cut-offs of G_i and G_{i + 1}. Generally, we set L_n = G_n (where n is the deepest level in the stack, or the coarsest, most blurred level). For each Laplacian level:

$$L_i = G_i - G_{i+1}$$

Where Laplacian stacks and Gaussian stacks really become handy is in multi-resolution blending!

Part 2.4: Multi-resolution Blending (AKA the oraple!)

Blending two images together involves strategically distorting them to create a smooth seam between them. If the blending window between the two images is too narrow, then

Contents

- Part 1.1: Convs from Scratch
- Part 1.2: Finite Difference Operators
- Part 1.3: Derivative of Gaussians Part
- 2.1: Image Sharpening
- Part 2.2: Hybrid Images
- Part 2.3: Gaussian and Laplacian Stacks
- Part 2.4: Multi-resolution Blending

the image spline will appear very sharp! On the other hand, if the blending window between the two images is too wide, then there may be ghost or duplicate artifacts in the transition region. In 1983, Burt and Adelson introduced a method for multi-resolution blending that involves creating Laplacian stacks of the images, blending the images at each level in the Laplacian stacks using a corresponding mask whose blending region matches that level's resolution scale, and recombining all the levels into one blended result.

To implement multi-resolution blending, I started with two images. I built Laplacian stacks from each image (which I will denote as L^a and L^b, and refer to individual levels within each stack as L^a_i). The 0-th level in each image's Laplacian stacks held the highest frequencies, and the n-th level in each image's Laplacian stacks held the lowest frequencies. I also created a mask of the same dimensions as my two images! I put 1's in the mask at coordinates in the final result where I wanted image 1 to be visible and 0's in the mask where I wanted image 2 to be visible. For the mask to create the right amount of "smoothing" at each level, I used my Gaussian stack implementation to create a Gaussian stack of the mask.

So now I have n levels, and 3 elements to blend together at each level: image 1's Laplacian representation at that level, image 2's Laplacian representation at that level, and the mask's Gaussian representation at that level. To combine them together, I use the mask as weights (multiplying the mask element-wise). So the combined result at each level is:

$$L_i^c = G_i \cdot L_i^a + (1 - Gi) \cdot L_i^b$$

And finally, to put all the puzzle pieces together into a single, multi-resolution-blended masterpiece, I just add all the combined Laplacian layers!

I wrote some code to visualize each level of the multi-resolution blending stack. The first column is the Laplacian stack for image 1, the second column is the Laplacian stack for image 2, the third column is the Gaussian mask at that level, and the fourth column is the combined result at that level. The final row represents all the rows

Contents

- Part 1.1: Convs from Scratch
- Part 1.2: Finite Difference Operators
- Part 1.3: Derivative of Gaussians Part
- 2.1: Image Sharpening
- Part 2.2: Hybrid Images
- Part 2.3: Gaussian and Laplacian Stacks
- Part 2.4: Multi-resolution Blending

above it "squashed" into a single image (AKA summing all the levels in a stack into a single image). One implementation detail for producing the Laplacian stack visuals: Laplacian levels can have negative values so to display them visually, I remapped their values to make the middle value in each channel of each Laplacian stack have pixel intensity 0.5.

Multi-resolution blending example 1: the oraple

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

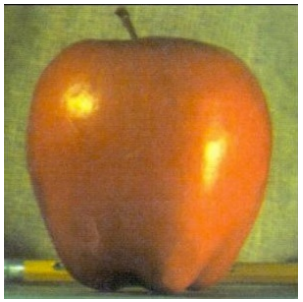
Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

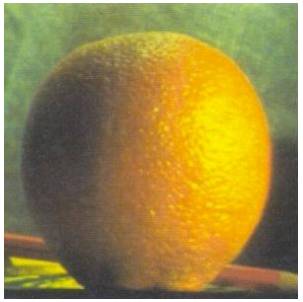
Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

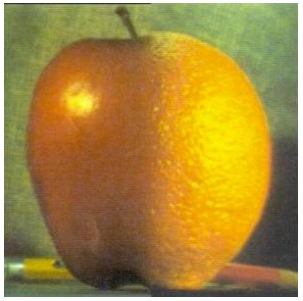
Part 2.4: Multi-resolution Blending



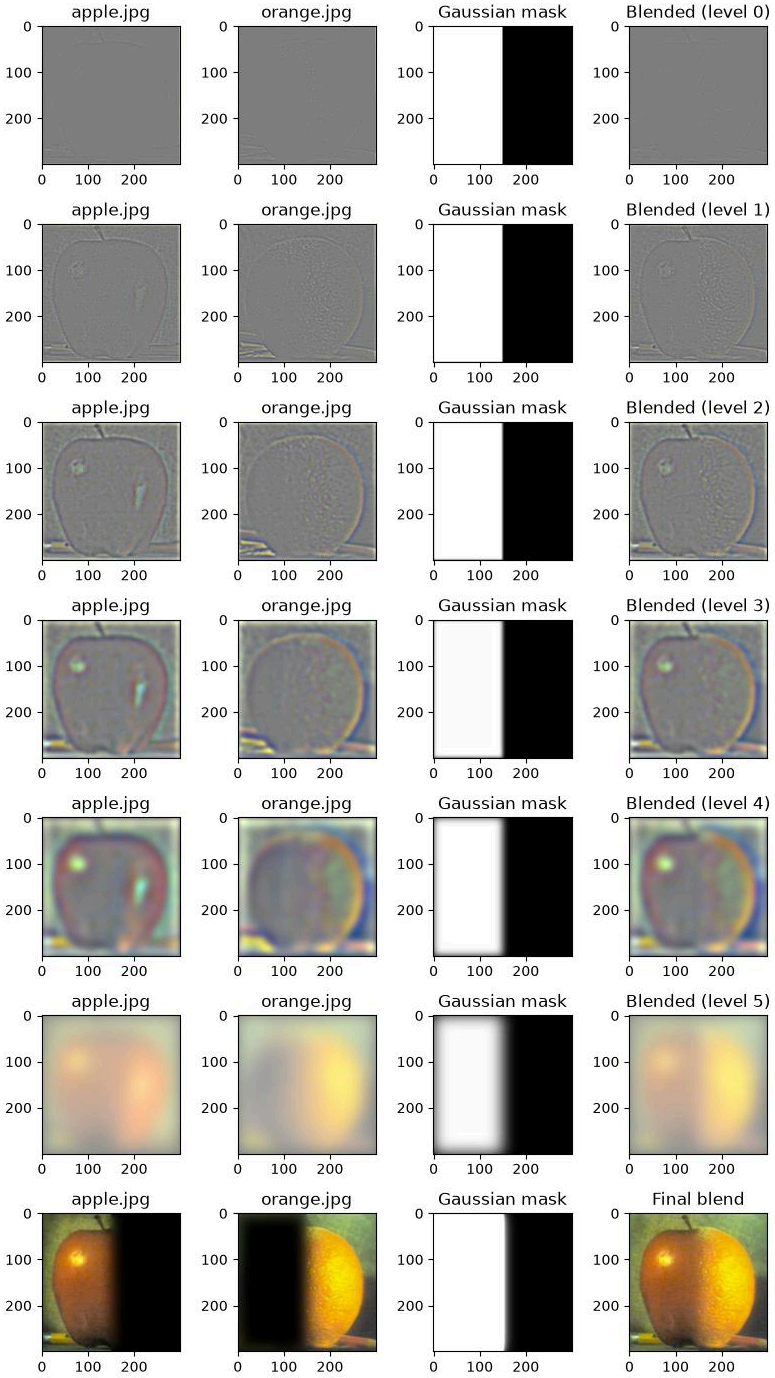
An apple



An orange



Boom: an oraple!



Visualizing each level of the Laplacian and Gaussian stacks for the oraple

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending

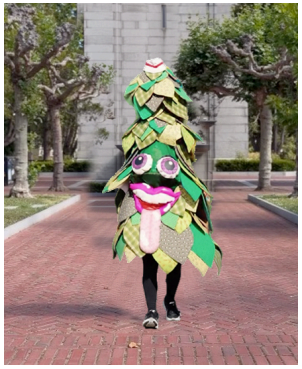
Multi-resolution blending example 2: Oski and the Stanford Tree, blended with a horizontal image spline

Contents

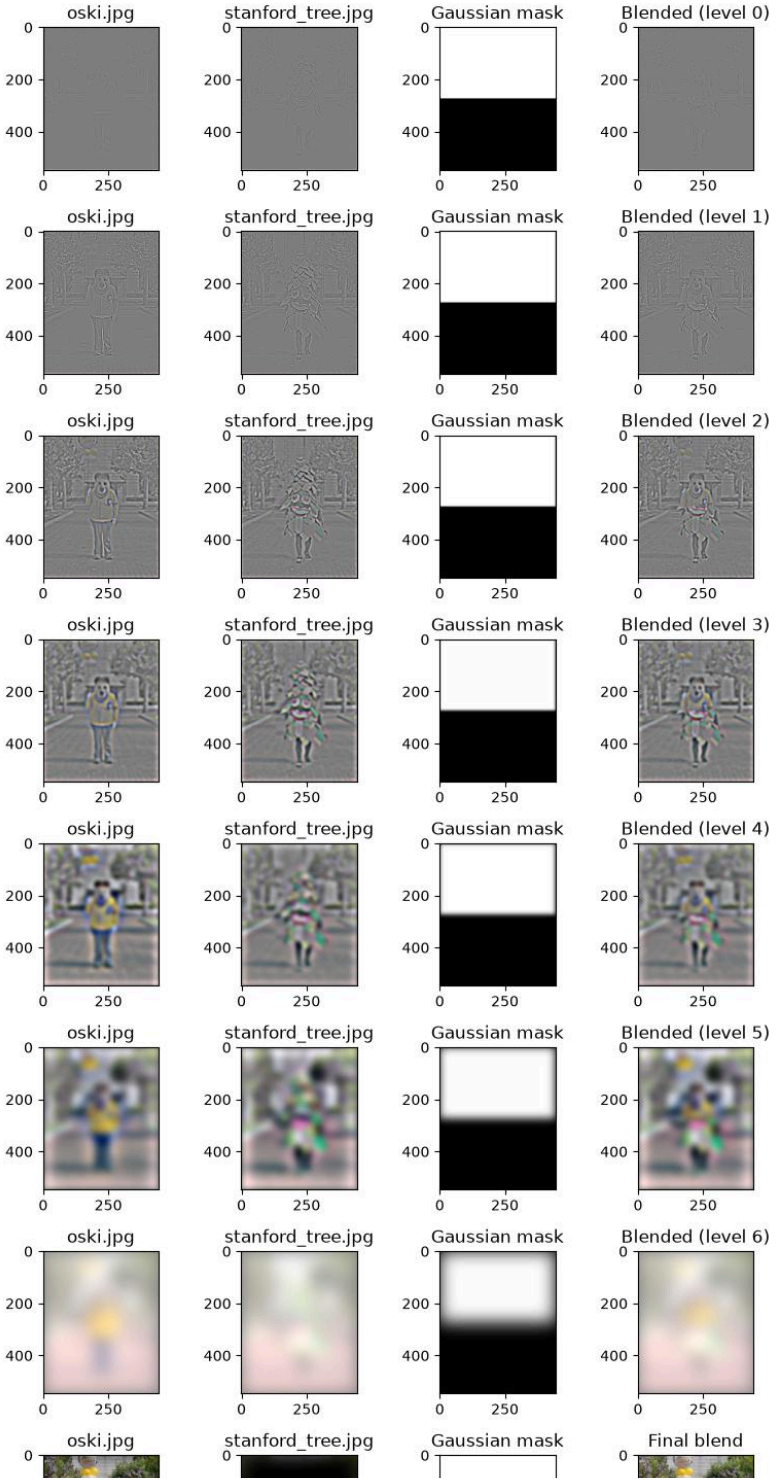
- Part 1.1: Convs from Scratch
- Part 1.2: Finite Difference Operators
- Part 1.3: Derivative of Gaussians Part
- 2.1: Image Sharpening
- Part 2.2: Hybrid Images
- Part 2.3: Gaussian and Laplacian Stacks
- Part 2.4: Multi-resolution Blending



Oski the bear



The Stanford Tree



Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

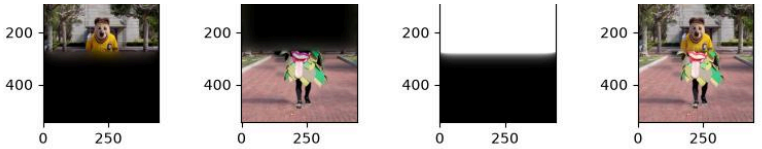
Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

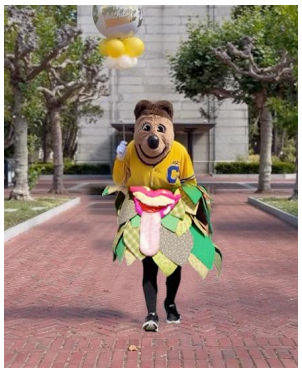
Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending



Visualizing each level of the Laplacian and Gaussian stacks for Oski and the Stanford Tree



Os-tree!

Multi-resolution blending example 3: a dinosaur nugget in Jurassic Park

I also implemented an interactive widget using `matplotlib.widgets.PolygonSelector` and `skimage.draw.polygon` which let me freely select points in an image, join them into the vertices of a polygon, and use the filled polygon region as a custom mask for multi-resolution blending. Using my custom mask creation function, I also blended together an image of a dinosaur chicken nugget and the iconic scene from Jurassic Park when Alan Grant and Ellie Sattler see live dinosaurs for the first time.

Contents

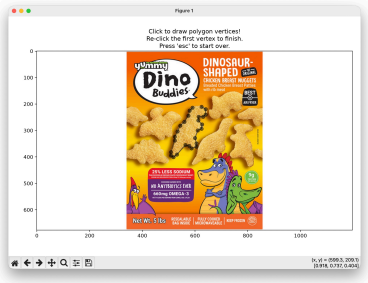
- Part 1.1: Convs from Scratch
- Part 1.2: Finite Difference Operators
- Part 1.3: Derivative of Gaussians Part
- 2.1: Image Sharpening
- Part 2.2: Hybrid Images
- Part 2.3: Gaussian and Laplacian Stacks
- Part 2.4: Multi-resolution Blending



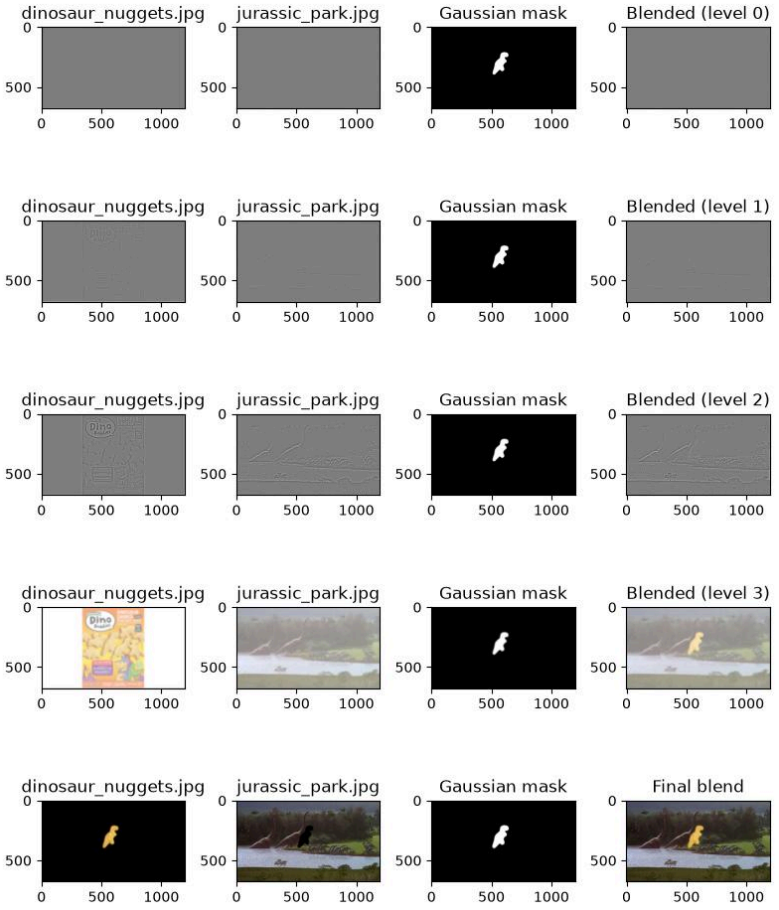
Dino Buddy dinosaur nuggets



The moment in Jurassic Park when Ellie and Alan see live dinosaurs for the first time



Selecting points to outline a dinosaur nugget and create a custom mask



Visualizing each level of the Laplacian and Gaussian stacks for dinosaur nuggets and Jurassic Park

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending



"Life, uh, finds a way"

Contents

Part 1.1: Convs from Scratch

Part 1.2: Finite Difference Operators

Part 1.3: Derivative of Gaussians Part

2.1: Image Sharpening

Part 2.2: Hybrid Images

Part 2.3: Gaussian and Laplacian Stacks

Part 2.4: Multi-resolution Blending